Patrones Creacionales

Patrón singleton

- Garantiza que una clase tenga una única instancia, proporcionando un método para la obtención de dicha instancia.

- Útil en aquellos casos en los que las instancias consumen muchos recursos y pueden ser compartidas por varios usuarios de la misma

- Ejemplo: las clases de configuración o de gestión de logging

- Al implementar este tipo de clases, tenemos que tener en cuenta que deberán tener constructor o constructores privados y un método estático y público para obtención de la instancia

Patrón singleton. Ejemplo práctico

- **Clase para registro de logs:**

```
public class LoggerSingleton {
    // Instancia única
    private static LoggerSingleton logger;
    // Constructor privado para evitar crear instancias desde el exterior
    private LoggerSingleton() {}
    // Crea o devuelve la instancia existente
    public static synchronized LoggerSingleton getInstance() {
        if (logger == null) {
            logger = new LoggerSingleton();
        }
        return logger;
    }
    // Método para registrar mensajes
    public void log(String mensaje) {
        System.out.println("[LOG] " + mensaje);
    }
}
```

Patrón singleton. Ejemplo práctico. Uso

```
public class UsoLogger {
    public static void main(String [] args){
        LoggerSingleton ls1=LoggerSingleton.getInstance();
        LoggerSingleton ls2=LoggerSingleton.getInstance();
        ls1.log("mensaje 1);
        ls2.log("mensaje 2);

    }
}
```

Misma instancia
LoggerSingleton

- **Muestra por pantalla:**

[LOG] mensaje 1
[LOG] mensaje 2

Patrón singleton. Versión Java moderna

➢Utilizando enum:

```
public enum LoggerSingleton {
    INSTANCE; //se crea una única instancia de forma segura, garantizado por la JVM
    public void log(String msg) {
        System.out.println("[LOG] " + msg);
    }
}
```

➢Utilización:

```
LoggerSingleton logger1 = LoggerSingleton.INSTANCE;
LoggerSingleton logger2 = LoggerSingleton.INSTANCE;
System.out.println(logger1==logger2);//true
```

- **Instancia única garantizada por la JVM**
- **Thread safe**
- **Garantiza única instancia durante deserialización**

Patrón Prototype

- **Clases que proporcionan métodos para clonar objetos de la misma a partir de un objeto prototipo**
- **Se emplea en aquellos contextos en los que se van trabajar con varios objetos iguales y donde la creación de los mismos a través de constructores resulta algo compleja.**
- **Habitualmente, una clase prototype implementa la interfaz Cloneable(no tiene métodos) y sobrescribe el método *clone()* heredado de Object, haciendo una llamada al *clone()* original.**

Patrón Prototype. Ejemplo práctico

- **Clase para clonar objetos Libro**

```
public class Libro implements Cloneable{
    private String titulo;
    private String isbn;
    private String autor;
    public Libro(String titulo, String isbn, String autor) {
        super();
        this.titulo = titulo;
        this.isbn = isbn;
        this.autor = autor;
    }
    //getter setter
    @Override (de Object)
    public Libro clone() {
        try {
            //llamada al método clone() heredado de Object
            return (Libro)super.clone();
        }catch(CloneNotSupportedException ex) {
            throw new AssertionError();
        }
    }
    @Override
    public String toString() {
        return "Libro [titulo=" + titulo + ", isbn=" + isbn + ", autor=" + autor + "]";
    }
}
```

Patrón Prototype. Ejemplo uso

- **A partir del prototipo se crea un clon:**

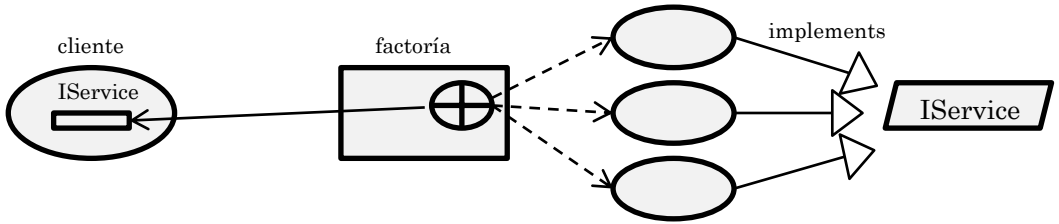
```
Libro l1=new Libro("Java 20", "11111A", "yo mismo");
Libro l2=l1.clone();//l2 apunta a un objeto idéntico a l1,
//pero son distintos objetos
l2.setAutor("tu mismo");
System.out.println(l1); //Libro [titulo=Java 20, isbn=11111A, autor=yo mismo]

System.out.println(l2); //Libro [titulo=Java 20, isbn=11111A, autor=tu mismo]
```

- **El método *clone()* solo hace copias superficiales, es decir, de los atributos de la clase. Si alguno de los atributos de la clase es de otra clase, ésta debe tener su propia implementación de *clone()*.**

Patrón Factory

- Delega la creación de objetos que implementen una interfaz en un método de una clase, conocida como clase de factoria, de modo que los clientes de esos objetos no tengan que conocer la clase a la que pertenecen
- Útil para el desacoplamiento entre capas, por ejemplo, entre capa service y DAO, o entre controller y service.
- El objetivo es que una clase cliente pueda utilizar distintas implementaciones de una interfaz sin ser modificada.



Patrón Factory. Ejemplo práctico

➢Interfaz Dao y posibles implementaciones:

```
public interface ProductosDao {
            Producto save(Producto producto);
            List<Producto> findByCategory(String category);
            :
}
```

```
public class ProductosDaoJdbc implements ProductosDao {
            //métodos
}
```

```
public class ProductosDaoJPA implements ProductosDao {
            //métodos
}
```

➢Factorías:

```
public interface
ProductosDaoFactory {
    ProductosDao create();
}
```

```
public class JdbcProductosDaoFactory implements
ProductosDaoFactory {
    @Override
    public ProductosDao create() {
        return new ProductosDaoJdbc();
    }
}
public class JpaProductosDaoFactory implements
ProductosDaoFactory {
    @Override
    public ProductosDao create() {
        return new ProductosDaoJpa();
    }
}
```

USO:

```
public class App {
    public static void
main(String[] args) {
        ProductosDaoFactory factory
= new JdbcProductosDaoFactory();
        ProductosDao dao =
factory.create();
    }
}
```

Patrón Factory. Ejemplo uso

- **Capa service independiente de la implementación:**

```
public class ProductosService {
    ProductosDao productosDao;
    //Trabaja contra la interfaz, no contra una clase concreta
    //cumple principios SOLID (segundo y quinto)
    public ProductosService(ProductosDao productosDao) {
        this.productosDao = productosDao;
    }
    public boolean guardar(Producto producto) {
        if(producto==null) {
            return false;
        }
        return productosDao.save(producto)!=null;
    }
    public List<Producto> buscarPorCategoria(String categoria) {
        return productosDao.findByCategory(categoria);
    }
}
```

```
//JDBC
ProductosDaoFactory factory = new JdbcProductosDaoFactory();
ProductosService serviceJdbc=new ProductosService(factory.create());
//JPA
factory=new JpaProductosDaoFactory();
ProductosService serviceJpa=new ProductosService(factory.create());
```

Patrón Abstract Factory

- Similar a Factory, pero cuando se debe tratar con familias de objetos.
- La interfaz de factoría define tantos métodos de creación como tipos de objetos.
- Por ejemplo, necesitamos generar implementaciones de ProductosDao y PedidosDao.
- La interfaz de factoría quedaría:

```
public interface DaoFactory {
    ProductosDao createProductosDao();
    PedidosDao createPedidosDao();
}
```

Patrón Abstract Factory

- **Las factorías de cada familia:**

```
public class JdbcDaoFactory implements DaoFactory {
    @Override
    public ProductosDao createProductosDao() {
        return new ProductosDaoJdbc();
    }

    @Override
    public PedidosDao createPedidosDao() {
        return new PedidosDaoJdbc();
    }
}

public class JpaDaoFactory implements DaoFactory {
    @Override
    public ProductosDao createProductosDao() {
        return new ProductosDaoJpa();
    }

    @Override
    public PedidosDao createPedidosDao() {
        return new PedidosDaoJpa();
    }
}
```

Patrón Abstract Factory

- **Ejemplo de uso:**

```
public class App {
    public static void main(String[] args) {
        // Aquí indicamos tipo de familia
        DaoFactory factory = new JpaDaoFactory();

        ProductosDao productosDao = factory.createProductosDao();
        PedidosDao pedidosDao = factory.createPedidosDao();

        productosDao.saveProducto("Pan");
        pedidosDao.savePedido("Pedido #123");
    }
}
```

Patrón Builder

- Consiste en utilizar un objeto simple para crear uno objeto completo, que puede requerir diferentes pasos para su creación
- Se utiliza en aquellos casos donde un objeto puede ser creado con diferentes parámetros, evitando tener múltiples constructores difíciles de manejar.
- En muchas ocasiones, el builder suele definirse como una clase interna dentro de la clase cuyos objetos se deben crear.

Patrón Builder. Ejemplo práctico

- **Clase Conector con varios atributos que se pueden configurar:**

```
public class Conector {
    private String modo;
    private int tipo;
    private String dir;
    private boolean estado;
    private String protocolo;
    //setter y getter

}
```

- **No sería viable, ni posible, tener múltiples constructores con todas las posibilidades a configurar**

Patrón Builder. Ejemplo práctico

- **Se crea una clase Builder anidada :**

```
public static class ConectorBuilder {
    Conector conector;
    public ConectorBuilder() {
        conector=new Conector();
    }
    public ConectorBuilder modo(String modo) {
        conector.setModo(modo);
        return this;
    }
    public ConectorBuilder tipo(int tipo) {
        if(tipo<0) {
            tipo=0;
        }
        conector.setTipo(tipo);
        return this;
    }
    public ConectorBuilder dir(String dir) {
        conector.setDir(dir);
        return this;
    }
    public ConectorBuilder estado(boolean estado) {
        conector.setEstado(estado);
        return this;
    }
    public ConectorBuilder protocolo(String protocolo) {
        conector.setProtocolo(protocolo);
        return this;
    }
    public Conector build() {
        return conector;
    }
}
```

Patrón Builder. Ejemplo uso

- **Ejemplo que crea un objeto Conector configurado a partir de tres atributos:**

```
Conector conector=new Conector.ConectorBuilder()
    .dir("prueba")
    .estado(true)
    .protocolo("http")
    .build();
```

- **Para crear objetos conector, llamamos a tantos métodos de builder como parámetros queramos configurar.**
- **La llamada final a *build()* devuelve el objeto creado.**

Patrón Builder objeto Inmutable

- **Permite que el objeto creado sea inmutable:**

```
public class Conector{
    private final String modo;
    private final int tipo;
    private final String dir;
    private final boolean estado;
    private final String protocolo;

    private Conector(Builder builder) {
        this.modo = builder.modo;
        this.tipo = builder.tipo;
        this.dir = builder.dir;
        this.estado = builder.estado;
        this.protocolo = builder.protocolo;
    }
}
```

Patrón Builder. Objeto inmutable

- **Se crea una clase Builder anidada :**

```
public static class Builder {
        private String modo;
        private int tipo;
        private String dir;
        private boolean estado;
        private String protocolo;
        public Builder modo(String modo) {
            this.modo = modo;
            return this;
        }
        public Builder tipo(int tipo) {
            this.tipo = tipo < 0 ? 0 : tipo;
            return this;
        }
        public Builder dir(String dir) {
            this.dir = dir;
            return this;
        }
        public Builder estado(boolean estado) {
            this.estado = estado;
            return this;
        }
        public Builder protocolo(String protocolo) {
            this.protocolo = protocolo;
            return this;
        }
        public Conector build() {
            return new Conector(this); // ⚠ Se construye solo al final
        }
    }
```